Prometheus Local Storage

A Complete Deep-Dive Reference — TSDB, WAL, Compaction, Retention, Remote Storage & Backfilling

---

Compiled study notes based on the official Prometheus documentation (prometheus.io/docs/prometheus/latest/storage/), expanded with explanations, diagrams, tables, and memory aids for easier learning.

WHAT THIS COVERS: *This document explains every concept from the official Storage page in detail — on-disk layout, WAL sizing, compaction mechanics, retention rules, remote storage integrations, and backfilling — with plain-English breakdowns alongside the technical facts.*

1. Local Storage — Overview

Prometheus ships with its own purpose-built time series database called **TSDB**. It stores everything on local disk in a custom, highly compressed format — no external database is required to run Prometheus. Optionally, Prometheus can also integrate with external/remote storage systems (covered in Section 6).

■ ***Mental model: TSDB is Prometheus's own built-in engine for writing, compressing, and querying time series data — it is not a generic database like MySQL or Postgres, it's optimized specifically for time-stamped numeric samples.***

2. On-Disk Layout

2.1 The 2-Hour Block Cycle

Incoming samples are grouped into **two-hour blocks**. Every block is a directory on disk containing:

- **chunks/** — the actual compressed time series sample data for that 2h window
- **index** — a file that maps metric names + labels to the correct series inside chunks/, enabling fast lookups without scanning everything
- **meta.json** — metadata about the block (time range, compaction lineage, stats)
- **tombstones** — records of series that were deleted via the API (data not actually removed yet — see 2.3)

2.2 Segment Files Inside chunks/

The chunks directory isn't one giant file — sample data is split into **segment files**, each capped at **512 MB by default**. This keeps individual files manageable and makes partial reads efficient.

2.3 Tombstones — Delete Markers, Not Real Deletes

When you delete series via the Prometheus Admin API, the data is **not** immediately erased from the chunk segments. Instead, a record is written to a separate **tombstones** file marking that series as deleted. The actual physical removal of that data only happens later, during **compaction** (Section 3), when the block gets rewritten anyway.

■ ***Mental model: Tombstone = a sticky note saying 'ignore this data' — the data is still physically on disk until the next compaction pass rewrites the block without it.***

2.4 The Head Block (In-Memory, Not Yet Persisted)

The **current**, still-being-written block lives in memory — it is not fully written to disk as a finished block yet (that only happens every 2 hours). Because in-memory data would be lost on a crash or restart, it's protected by the **Write-Ahead Log (WAL)** — see Section 2.6.

2.5 Example Data Directory Structure

```
./data
+-- 01BKGV7JBM69T2G1BGBGM6KB12
|   +-- meta.json
+-- 01BKGTZQ1SYQJTR4PB43C8PD98
|   +-- chunks
|   |   +-- 000001
|   +-- tombstones
|   +-- index
|   +-- meta.json
+-- 01BKGTZQ1HHWHV8FBJXW1Y3W0K
|   +-- meta.json
+-- 01BKGV7JC0RY8A6MACW02A2PJD
|   +-- chunks
|   |   +-- 000001
|   +-- tombstones
|   +-- index
|   +-- meta.json
+-- chunks_head
|   +-- 000001
+-- wal
    +-- 000000002
    +-- checkpoint.00000001
        +-- 00000000
```

Notice: the long alphanumeric folder names (e.g. `01BKGV7JBM...`) are ULIDs — unique block identifiers. Each is an independent, self-contained block. `chunks_head/` holds memory-mapped head chunks, and `wal/` holds the write-ahead log segments.

2.6 Write-Ahead Log (WAL) — Full Detail

The WAL exists purely for **crash recovery**. If Prometheus restarts before the current 2h block is flushed to disk, it replays the WAL to rebuild the in-memory head block exactly as it was.

Property	Value / Behavior
Segment file size	128 MB each
Minimum files retained	3 segment files, always
Guaranteed time coverage	At least 2 hours of raw (uncompacted) data
High-traffic servers	Automatically retain **more than 3** files if needed, to still guarantee the 2h coverage
Contains	Raw, uncompacted data — so WAL files are noticeably larger per-sample than regular compacted block files
Compression flag	`--storage.tsdb.wal-compression` — roughly halves WAL size for a small CPU cost. Default ON since v2.20.0 (introduced in v2.11.0)

Property	Value / Behavior
Downgrade warning	Once WAL compression is enabled, downgrading below v2.11.0 requires deleting the WAL

Why exactly 2 hours? Because that matches the block-flush cycle — WAL only needs to cover the gap between the last completed block and right now. Once a block is flushed, the WAL segments covering that period become redundant and get cleaned up via checkpointing (you can see this in the example layout above: `checkpoint.00000001`).

■ ***Mental model: WAL size is NOT fixed — it's driven by TIME (2h coverage guarantee), not a fixed file count. Low traffic → small WAL. High traffic → more/larger segments kept automatically to preserve that same 2-hour safety window.***

2.7 Local Storage Limitation: No Clustering

LIMITATION: *Local storage is NOT clustered or replicated. It cannot scale arbitrarily and is not durable against drive/node failure — treat it like any other single-node database. That said, with proper capacity planning it's entirely possible to retain years of data locally.*

2.8 Backups — Snapshots Recommended

Prometheus exposes a **Snapshot API** for safe backups. Backing up files directly without a snapshot risks losing recent data — specifically, anything recorded since the last completed block (which covers roughly the last **3 hours** of samples, since blocks flush every 2h but WAL retains a bit more).

If you back up or restore while **excluding** the WAL-related directories (`chunks_head/`, `wal/`, `wbl/`), the backup will still be internally coherent — you simply lose the time range those directories cover.

2.9 Filesystem Requirements

This is a hard operational constraint, not a suggestion:

CAUTION — DO NOT USE NFS/EFS: *Non-POSIX-compliant filesystems are NOT supported for Prometheus local storage — unrecoverable corruption can happen. This explicitly includes NFS (Network File System), including AWS EFS. Even though NFS can theoretically be POSIX-compliant, most real-world implementations are not. Always use a local filesystem for reliability.*

2.10 Recovering from Corrupted Storage

If local storage becomes corrupted to the point Prometheus won't even start:

- **Best option:** back up the storage directory, then restore the corrupted block directories from your own prior backups
- **Last resort (no backups available):** manually remove the corrupted files — e.g. delete individual block directories, or delete the WAL files entirely

- **Trade-off:** removing files this way means permanently losing the data for whatever time range those specific blocks or WAL segments covered

3. Compaction

3.1 What Compaction Does

The initial 2-hour blocks are eventually merged together in the background into larger, more efficient blocks. This reduces the number of files Prometheus needs to open to answer a query spanning a long time range, and improves compression by working over a bigger, more uniform dataset.

3.2 The Size Cap Rule

RULE: *Compaction creates larger blocks containing data spanning up to* ***10% of the configured retention time, OR 31 days — whichever is smaller****. So compaction doesn't just merge everything into one giant block forever; it progressively builds bigger blocks up to that cap.*

Example: if your retention is 90 days, 10% = 9 days, and since 9 days < 31 days, blocks will compact up to a maximum span of 9 days. If retention is 400 days, 10% = 40 days, but since 31 days is smaller, the cap is 31 days.

3.3 Temporary Disk Overshoot During Compaction

Because both the **source blocks** (the small ones being merged) and the **newly compacted block** must exist on disk *simultaneously* while compaction runs, your on-disk size can briefly exceed the configured `storage.tsdb.retention.size` limit. This excess is released only once the next retention cleanup pass removes the old source blocks.

■ ***Mental model: Compaction never modifies blocks in place — it builds the new merged block fully first, verifies it, and only then deletes the old source blocks. This atomic build-then-delete pattern is what makes it crash-safe: if Prometheus dies mid-compaction, nothing is lost because the originals are still intact.***

4. Operational Flags (Configuration)

These are the key command-line flags that control local storage behavior:

Flag	What It Does
--storage.tsdb.path	Where Prometheus writes its database. Default: `data/`
--storage.tsdb.retention.time	How long to retain samples. Default **15d** if neither this nor retention.size is set. Units: y, w, d, h, m, s, ms
--storage.tsdb.retention.size	Max total bytes of storage blocks to retain. Oldest data removed first. Default: 0 (disabled). Units: B, KB, MB, GB, TB, PB, EB (powers-of-2 based, so 1KB = 1024B)
--storage.tsdb.wal-compression	Enables WAL compression — roughly halves WAL size for a little extra CPU. Default ON since v2.20.0

4.1 Important Nuance on retention.size

DISK SIZING GOTCHA: *Only **persistent blocks** are deleted to honor `retention.size` — but the WAL and memory-mapped head chunks (`chunks_head/`) are still COUNTED toward the total size, even though they aren't deleted by this setting. This means your minimum required disk space = the peak space taken by WAL+Checkpoint and chunks_head combined (which peaks every 2 hours) — on top of whatever retention.size you configure.*

4.2 Retention: Time vs. Size — Whichever Triggers First

If **both** `retention.time` and `retention.size` are configured, Prometheus applies **whichever limit is hit first**.

4.3 Expired Block Cleanup Timing

TIMING: *Expired block cleanup runs in the background and may take **up to 2 hours** to actually remove expired blocks. A block must be **fully expired** (every sample inside it past the retention cutoff) before it is removed — no partial trimming of a block happens here.*

4.4 Capacity Planning Formula

Prometheus stores an average of only **1–2 bytes per sample** thanks to its compression. Use this rough formula to plan disk capacity:

```
needed_disk_space = retention_time_seconds
                  × ingested_samples_per_second
                  × bytes_per_sample
```

To reduce ingestion rate, you have two levers:

- Reduce the number of time series scraped (fewer targets, or fewer series per target)

- Increase the scrape interval (scrape less often)

> ***TIP:*** *Reducing the* ***number of series*** *is generally more effective than increasing scrape interval, because compression works within a single series over time — fewer series means less overhead overall, whereas a longer interval just spreads the same series thinner.*

5. Right-Sizing Retention Size

If you're using `storage.tsdb.retention.size` to cap disk usage, don't set it to exactly your available disk space — leave a buffer.

RECOMMENDATION: *Recommended: set retention.size to* ***at most 80–85%*** *of your total allocated Prometheus disk space. The remaining* ***15–20% buffer*** *exists specifically to absorb the temporary extra space that in-progress compactions need (source blocks + new compacted block coexisting on disk, as covered in Section 3.3) — before the old blocks get cleaned up.*

■ ***Mental model: Think of the 15–20% buffer as compaction's 'elbow room' — without it, compaction could push you over your actual disk limit and cause real out-of-space failures, not just a configured-limit overshoot.***

6. Remote Storage Integrations

Prometheus's local storage is inherently limited to a single node — it doesn't try to solve clustered/distributed storage itself. Instead, it exposes interfaces so it can integrate with dedicated remote storage systems (like Thanos, Cortex, Mimir, and others).

6.1 The Four Integration Directions

- **Remote Write (as client):** Prometheus writes/pushes ingested samples out to a remote URL
- **Remote Write (as server):** Prometheus can receive samples pushed to it from other clients
- **Remote Read (as client):** Prometheus reads sample data back from a remote URL
- **Remote Read (as server):** Prometheus returns sample data when other clients request it

6.2 Protocol Details

Aspect	Detail
Encoding	Snappy-compressed Protocol Buffers, over HTTP
Read protocol stability	NOT yet considered a stable API
Write protocol stability	v1.0 spec is STABLE; v2.0 spec is experimental. Prometheus server supports both
Enable receiving remote writes	Flag: `--web.enable-remote-write-receiver` → endpoint becomes `/api/v1/write`
Remote read endpoint	Always available at `/api/v1/read`

6.3 Important Limitation of Remote Read

DESIGN LIMITATION: *On the read path, Prometheus only fetches **raw** series data (matching label selectors + time range) from the remote end. ALL PromQL evaluation still happens locally, inside the querying Prometheus server itself — the remote store does no computation. This means the entire matching dataset must be pulled into the local Prometheus before it can be processed, which creates a scalability ceiling. Fully distributed PromQL evaluation was deemed infeasible for now.*

■ ***Mental model: Remote read is a dumb pipe for raw data — the 'thinking' (PromQL) always happens on your own Prometheus server, never on the remote storage side. This is exactly why systems like Thanos add their own Querier layer instead of relying purely on Prometheus's built-in remote read.***

7. Backfilling from OpenMetrics Format

7.1 What Backfilling Is For

Backfilling lets you create TSDB blocks directly from historical data that's in **OpenMetrics format** — typically used to migrate metrics data from a different monitoring system into Prometheus.

> ***CAUTION:*** *It is NOT safe to backfill data from the last 3 hours — that time range may overlap with the current head block Prometheus is still actively mutating in memory, which can cause conflicts.*

Each backfilled block covers 2 hours of data by default (same as normal ingestion), which keeps memory requirements for block creation bounded. Prometheus itself later compacts these 2-hour blocks into larger ones, exactly like normally-ingested data.

> ***UNSUPPORTED FEATURES:*** *Native histograms and staleness markers are NOT supported by the backfilling procedure, because they cannot be represented in OpenMetrics format.*

7.2 Usage

```
promtool tsdb create-blocks-from openmetrics <input file> [<output directory>]
```

Default output directory: `./data/`. After block creation, move the resulting blocks into Prometheus's real data directory. If the new blocks overlap with existing ones, on Prometheus v2.38 and below you must also set `--storage.tsdb.allow-overlapping-blocks`. Backfilled data is still subject to your configured retention (time or size) like any other data.

7.3 Longer Block Durations for Backfilling

By default, promtool uses the standard 2h block duration — the safest, most generally correct choice. For backfilling a very long historical range, though, using a larger block duration can backfill faster and avoid extra compaction work later. Control this with:

```
--max-block-duration=<duration>
```

The backfilling tool then picks a suitable block duration no larger than this value.

> ***TRADE-OFF — USE WITH CARE:*** *Larger blocks come with real drawbacks: time-based retention must keep an ENTIRE block around if even one sample inside it is still within the retention window (even if most of the block is way past it) — wasting disk. Conversely, size-based retention will drop the WHOLE block the moment the size limit is crossed, even if it's only slightly over. For this reason, using fewer/larger blocks via a large max-block-duration is NOT recommended for production instances.*

8. Backfilling for Recording Rules

8.1 The Problem It Solves

When you create a new recording rule, it has no historical data — it only starts producing values from the moment it's created onward. promtool can generate historical data for a recording rule retroactively, computed against your existing raw metrics.

8.2 Usage Example

```
promtool tsdb create-blocks-from rules \
    --start 1617079873 \
    --end 1617097873 \
    --url http://mypromserver.com:9090 \
    rules.yaml rules2.yaml
```

The rule files must be normal Prometheus recording rules files. The command's output (default: `data/`) contains blocks with historical rule data for every rule in those files. Move these blocks into a running Prometheus instance's `storage.tsdb.path` directory to use them — on v2.38 and below, you again need `--storage.tsdb.allow-overlapping-blocks`. Once moved, the new blocks merge with existing ones on the next compaction run.

8.3 Limitations

- Running the rule backfiller multiple times with overlapping start/end times creates duplicate blocks containing the same data each run
- ALL rules in the provided files get evaluated — no selective filtering
- If a rule file sets its own `interval`, that takes priority over the `--eval-interval` flag passed to the backfill command
- Alerts present in the recording rule file are currently ignored entirely
- Rules within the same group CANNOT see results of previous rules in that group — dependent rule chains aren't supported. Workaround: backfill multiple times, creating dependency data first and moving it into the server before backfilling rules that depend on it

9. Quick Reference — Everything at a Glance

Concept	Key Fact
Block interval	Every 2 hours, head block → immutable disk block
Block contents	chunks/ (data), index (lookup), meta.json (metadata), tombstones (delete markers)
Segment file cap	512 MB per chunk segment file (default)
WAL segment size	128 MB each
WAL minimum retention	3 files, or more to guarantee ≥2h raw data coverage
Default retention.time	15 days (if neither time nor size retention set)
Compaction size cap	min(10% of retention time, 31 days)
Retention conflict rule	Whichever of time/size triggers first wins
Expired block cleanup delay	Up to 2 hours in the background
Recommended retention.size buffer	Set to 80–85% of allocated disk; 15–20% buffer for compaction overshoot
Bytes per sample	~1–2 bytes average, thanks to compression
Filesystem requirement	POSIX-compliant local filesystem only — NFS/EFS unsupported
Backfill danger zone	Do not backfill the last 3 hours (overlaps live head block)
Remote read limitation	Only raw data is fetched remotely; PromQL evaluation always happens locally

■ ***Mental model: The whole local-storage story in one sentence: samples land in memory (protected by WAL) → every 2h they become an immutable disk block → compaction periodically merges blocks up to a size cap → retention rules (time or size, whichever hits first) delete whole expired blocks → and if you need more than one node's worth of durability or scale, you hand data off to remote storage (like Thanos) since Prometheus deliberately does not solve clustering itself.***

Source: Prometheus official documentation, Storage page (v3.14). Compiled and expanded for study purposes.